

“CAPACITY CONNECT”
An Intelligent Peer-to-Peer Campus Skill Exchange Platform

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

MARRI HRUTHIKA 2520090002

K. GAYATHRI SRIVALLI 2520080010

A. YUKTHA 2520090093

Under the Esteemed Guidance of

Dr .Khaja Shareef

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled “**Capacity Connect – An Intelligent Peer-to-Peer Campus Skill Exchange Platform**” is a record of Bonafide work of MARRI HRUTHIKA – 2520090002, K. GAYATHRI SRIVALLI - 2520080010, A. YUKTHA – 2520090093 submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 12, Team 4, during 2026–2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

MARRI HRUTHIKA –2520090002

K.GAYATHRI SRIVALLI-2520080010

A.YUKTHA-2520090093

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled “**Capacity Connect – An Intelligent Peer-to-Peer Campus Skill Exchange Platform**” is a bonafide work done and submitted by **MARRI HRUTHIKA – 2520090002**, **K. GAYATHRI SRIVALLI – 2520080010**, **A. YUKTHA – 2520090093** in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 12, Team 4, at KLH University during 2026–2027, Trimester 4.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator, and my guide **Dr.Khaja Shareef**, Department of Computer Science and Information Technology which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Kaja Shareef**, Head of the ***Department for Computer Science and Information Technology*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

MARRI HRUTHIKA– 2520090002

K.GAYATHRI SRIVALLI-2520080010

A.YUKTHA-2520090093

Abstract

Capacity Connect is a web-based peer-to-peer learning platform designed to help students discover, share, and exchange skills within a campus community. Students often possess different technical, academic, creative, and practical skills but may face difficulty finding suitable peers who can teach the skills they want to learn. At the same time, students with valuable skills may not have an organized platform through which they can share their knowledge with others. These situations can limit collaborative learning and reduce opportunities for students to develop skills through peer interaction.

The proposed system provides a user-friendly platform through which students can create profiles, list the skills they can teach, specify the skills they want to learn, and discover suitable peers based on their interests and capabilities. Students can search for skills, view relevant profiles, send learning or teaching requests, and communicate with connected peers. The platform also supports reviews and ratings to encourage trustworthy interactions. Badges and a ranking mechanism provide recognition for active participation and encourage students to contribute to the campus learning community.

The application is designed as a centralized skill-exchange platform that connects **learners and mentors within the same student community**. The system organizes user profiles, skills, learning interests, requests, connections, communication, reviews, ratings, and achievements to provide a structured peer-learning environment. By enabling students to act as both learners and mentors, Capacity Connect promotes collaborative knowledge sharing, improves accessibility to peer-based learning, and transforms students from passive learners into active contributors within the campus community.

Index

S.No.	Chapters	Topics	Page No.
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience	5
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	7
3	Technology Stack	3.1 Frontend 3.2 Backend 3.3 Database 3.4 Version Control 3.5 Authentication and Security Libraries	8
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of Each Layer 4.3 Deployment Architecture	9
5	Design	5.1 Database Design Principles 5.2 ER Diagram / Database Schema 5.3 Design Decisions and Assumptions	10
6	Implementation	6.1 Module-wise Implementation 6.2 Front-end Logic 6.3 API Integration 6.4 Authentication and Authorization	12
7	Features	7.1 List of Main Features 7.2 How Each Feature Works	13
8	Testing	8.1 Postman API Testing 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results	15
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting of Frontend and Backend	17
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations or Known Bugs	18

S.No.	Chapters	Topics	Page No.
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimizations	19
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned	20
13	References	– Documentation, tools, and resources used	21
14	Appendices	– Sample Code Snippets – Installation / Setup Instructions – Default Login Credentials – API Reference Summary – Screenshots (to be added by the student)	22

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Students in a college campus possess a wide range of technical, academic, creative, and practical skills. However, students often face difficulty finding suitable peers who can teach the skills they want to learn. At the same time, students who already possess useful skills may not have a structured platform through which they can share their knowledge with others. This limits opportunities for collaborative learning and peer-to-peer knowledge exchange.

Capacity Connect was developed to address this gap by providing a centralized platform for campus-based skill exchange. Students can create profiles containing the skills they can teach and the skills they want to learn. The platform helps students discover suitable peers, connect with them, exchange knowledge, and build a collaborative learning community within the campus.

1.2 Problem Statement

Given a college campus with a fixed number of rooms of varying capacity and type, and a continuous stream of events being scheduled by different organizers, the system must answer three questions reliably and in real time: which rooms are free for a given date and time, whether a specific room can be safely assigned to a specific event, and whether a specific student can be allowed to register for a specific event. Answering these questions manually does not scale, is error-prone, and gives students no visibility into room availability until they physically check.

1.3 Scope of the Project

The system covers the full lifecycle of a campus event from the room's perspective: adding and maintaining a directory of rooms (with capacity, type, equipment, and status); creating and editing events; assigning a room to an event only after checking its status, capacity, and existing schedule; students registering for events subject to duplicate and capacity checks; an audit trail of booking actions; and a simple occupancy-tracking mechanism for recording actual attendance after an event takes place. Payment processing, multi-campus support, and a native mobile application were treated as out of scope for this submission.

1.4 Objectives

- Provide real-time room availability for any requested date and time window.
- Prevent a room from ever being double-booked, using a single, consistently-applied overlap rule.
- Enforce registration rules: no duplicate registrations, and no registrations beyond a room's capacity once one is assigned.
- Give administrators a single dashboard to manage rooms, events, bookings, registrations, and occupancy.
- Automatically calculate an occupancy percentage and status (Normal / High / Near Capacity / Over Capacity) for completed events.

- Secure the system with role-based access, so student accounts can never reach admin-only functionality.

1.5 Importance of the Application

By centralizing room and event data, the system removes the guesswork from room allocation. Administrators gain a live view of which rooms are occupied at any given moment and which upcoming events have a capacity mismatch, letting them intervene before an event happens rather than discovering a problem on the day. Students gain the ability to check room availability and register for events themselves, without needing to contact an administrator directly.

1.6 Target Users / Audience

The system serves two roles. Students use it to browse and search events, view event details, register for events, track their own registrations, and check room availability. Administrators use it to manage the room inventory, create and edit events, assign rooms to events, monitor all bookings and registrations across the campus, and track occupancy after events conclude.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The application has modest hardware requirements, since both the Node.js backend and the MySQL database are lightweight relative to the small dataset a single campus generates. A development or demonstration machine with a dual-core processor, 4 GB of RAM, and roughly 500 MB of free disk space (for Node modules, MySQL data files, and project source) is sufficient to run the entire stack — frontend, backend, and database — on one computer.

2.2 Software Requirements

- Node.js 18 or later, with npm
- MySQL 8.0 or later
- A modern web browser (Chrome, Edge, or Firefox)
- Git, for version control
- MySQL Workbench and Visual Studio Code, for development (optional but recommended)

2.3 Development Tools and Frameworks

React.js (bootstrapped with Vite) was used for the frontend; Node.js with the Express.js framework for the backend REST API; MySQL as the relational database, accessed through the mysql2 driver; Postman for API testing and documentation; and Git with GitHub for source control. VS Code was the primary development environment, and MySQL Workbench was used for inspecting and verifying the database schema and data directly.

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend

The frontend is a single-page application built with React.js and bundled using Vite. React Router handles client-side navigation between the public (login/register), student, and admin areas, and Axios is used as the single HTTP client for every API call, wrapped in one central service module per backend resource (rooms, events, registrations, bookings, occupancy). Styling is plain CSS with a small set of reusable design tokens (colors, spacing, card/table/badge styles) rather than a heavyweight UI framework, keeping the codebase easy to read and modify.

3.2 Backend

The backend is a REST API built with Node.js and Express.js, organized into routes, controllers, and models — routes declare the URL and required role, controllers hold the request/response and validation logic, and models hold the raw SQL. This separation keeps each file focused on one responsibility and makes the codebase straightforward to navigate and explain.

3.3 Database

MySQL 8 stores all persistent data across seven related tables (users, rooms, events, registrations, room_schedules, bookings, and occupancy_records). Referential integrity is enforced with foreign keys, uniqueness with UNIQUE constraints (e.g. one email per user, one registration per student per event), and data validity with CHECK constraints (e.g. no negative capacity, start time before end time).

3.4 Version Control

Git was used throughout development for version control, with the project structured as a single repository containing both the client and server codebases, hosted on GitHub.

3.5 Authentication and Security Libraries

Passwords are hashed with bcrypt (10 salt rounds) before being stored — plain-text passwords are never written to the database. Login sessions are stateless JSON Web Tokens (JWT), signed with a server-side secret and carrying only a user's ID, role, and name. Express middleware verifies this token on every protected route and enforces role-based access, ensuring student accounts cannot reach admin-only endpoints even if a request is crafted by hand.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The system follows a standard three-tier architecture, cleanly separating the user interface, the business logic, and the persisted data. Figure 4.1 shows the three layers and how a request flows between them.



Figure 4.1: High-level three-tier architecture of the Event and Room Occupancy Management System

4.2 Description of Each Layer

The Presentation Layer is the React single-page application. It never talks to MySQL directly; every piece of data it shows comes from calling the REST API over HTTP. During development, Vite's built-in proxy forwards any request the frontend makes to “/api/...” straight through to the Express server, so the browser never needs to know the backend's actual port or address.

The Application Layer is the Express.js REST API. Incoming requests pass through authentication middleware (which verifies the JWT and attaches the requesting user's identity and role), then into a controller that validates the request and applies business rules — for example, checking a room's status, capacity, and existing schedule before allowing it to be assigned to an event — before finally reading or writing data through a model.

The Data Layer is the MySQL database, accessed through a connection pool (via the `mysql2` driver) rather than a single long-lived connection, so multiple requests can be served concurrently without waiting on each other unnecessarily.

4.3 Deployment Architecture

For development and demonstration purposes, all three layers run as separate local processes on a single machine: the Express server listens on port 5000, the Vite development server serves the React app on port 5173 and proxies API calls to it, and MySQL listens on its default port 3306. No containerization or CI/CD pipeline was configured for this submission, since the course scope did not call for cloud deployment; Chapter 11 (Future Enhancements) outlines how this could be extended to a hosted environment.

CHAPTER 5: DESIGN

5.1 Database Design Principles

The schema was normalized to remove redundant data: each fact is stored in exactly one place, and every relationship between entities is expressed through a foreign key rather than by duplicating data across tables. Every table enforces its own invariants directly in the database — not just in application code — using PRIMARY KEY, FOREIGN KEY, UNIQUE, and CHECK constraints, so that invalid data (a negative capacity, an end time before a start time, a duplicate registration) is rejected at the database level even if a bug elsewhere in the application tried to insert it.

5.2 ER Diagram / Database Schema

The database consists of seven related tables, shown in Figure 5.1.

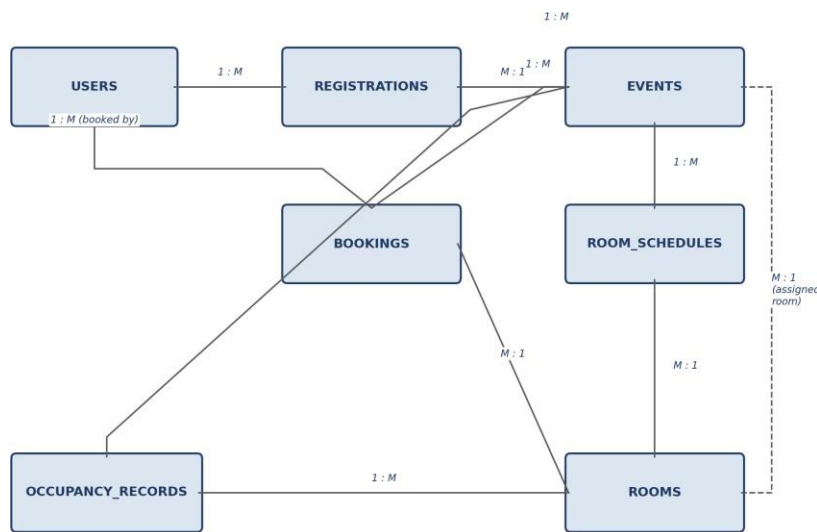


Figure 5.1: Entity-Relationship diagram of the 7-table database schema

USERS stores every account (student or admin) with a bcrypt-hashed password and a role flag. ROOMS holds the physical inventory of bookable spaces — name, number, building, capacity, type, equipment, and status. EVENTS holds each event's details and an optional assigned room. REGISTRATIONS links a user to an event they have signed up for, with a UNIQUE constraint preventing the same student registering twice for the same event. ROOM_SCHEDULES holds the authoritative time-block for every room-event pairing and is the single table every conflict check reads from. BOOKINGS records who (which admin) made a given room assignment and when, serving as an audit trail rather than a second source of truth for conflict-checking. OCCUPANCY_RECORDS stores expected, registered, and actual attendance figures per event, room, and date.

5.3 Design Decisions and Assumptions

Two design decisions were made explicitly, where the original entity list left some ambiguity, and are recorded here rather than left implicit in the code:

- ROOM_SCHEDULES vs. BOOKINGS: since both tables could plausibly record “room + time,” ROOM_SCHEDULES was designated the single source of truth that every availability check and every room-assignment reads from, while BOOKINGS is treated purely as an audit trail of who performed the assignment. Both are written together, atomically, inside one database transaction, so they can never drift out of sync.
- Effective registration capacity: the EVENTS table has no dedicated “maximum participants” field, so once a room is assigned to an event, that room's capacity is treated as the effective registration limit. Before a room is assigned, registration is left uncapped.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

The backend is organized into six resource modules, each with its own routes, controller, and model file: Authentication, Rooms, Events, Registrations, Bookings, and Occupancy. The most involved piece of logic sits in the Events module's room-assignment function, which runs four checks in sequence — the room exists and is active, its capacity is sufficient for the event's expected participants, and it has no conflicting schedule for the requested time — and only if all four pass does it atomically update the event's assigned room, write a new room_schedules row, and log a bookings row, inside a single database transaction so a failure partway through leaves no partial state behind.

The core conflict-detection rule used throughout the system is: a requested time slot conflicts with an existing schedule if $(\text{existing_start} < \text{requested_end}) \text{ AND } (\text{existing_end} > \text{requested_start})$. This single rule is applied consistently by the room-availability search, the event room-assignment workflow, and the booking-creation endpoint, so a room can never end up double-booked no matter which entry point was used to book it.

6.2 Front-end Logic

The frontend uses React's Context API for the two pieces of state that genuinely need to be shared across the whole application — the logged-in user (AuthContext) and toast notifications (ToastContext) — and otherwise keeps state local to the component that needs it. This was a deliberate choice to avoid the added complexity of a state-management library such as Redux, which was not necessary at this project's scale. Shared UI elements (a Modal, a ConfirmDialog, status badges, and loading/empty/error placeholders) are each written once and reused across every admin and student page, so every page in the app fails, loads, and confirms actions in a visually consistent way.

6.3 API Integration

All HTTP calls go through one central Axios instance, which automatically attaches the logged-in user's JWT to every outgoing request and automatically logs the user out if any response comes back with a 401 Unauthorized status (for example, after the token expires). Each backend resource has a matching frontend “service” file (roomService.js, eventService.js, and so on) so that no page ever hardcodes a URL or repeats the same fetch logic.

6.4 Authentication and Authorization

Registration always creates a student account — there is deliberately no way to create an admin account through the public API, so an attacker (or a curious student) cannot register their way into administrator access. Admin accounts exist only because they were inserted directly into the database. Every protected route runs a `protect` middleware that verifies the JWT, and admin-only routes additionally run an `adminOnly` middleware that checks the token's role claim, rejecting any student token with a 403 Forbidden response.

CHAPTER 7: FEATURES

7.1 List of Main Features

Student-facing features:

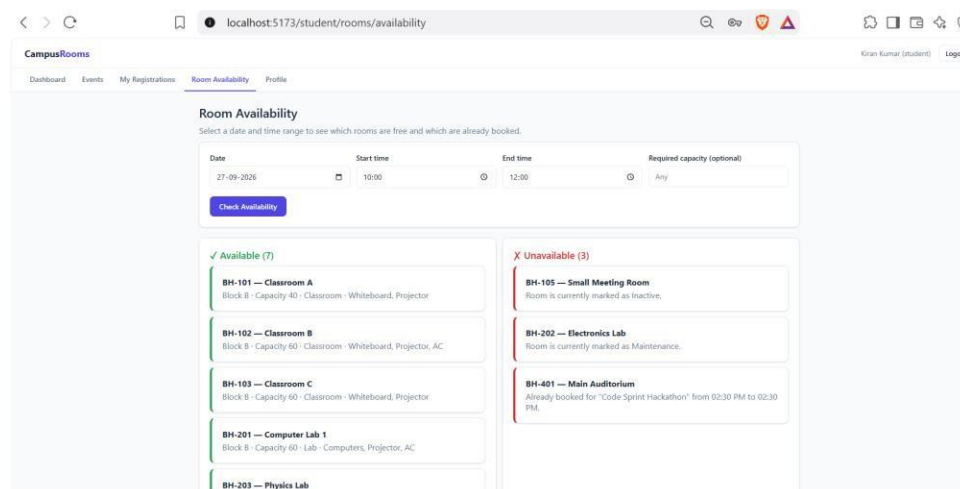
- Browse, search, and filter upcoming events by name and type
- View full event details, including assigned room and capacity
- Register for an event, with duplicate-registration and capacity limits enforced automatically
- View “My Registrations” with live status (Registered / Cancelled / Attended)
- Check room availability for any date and time window
- View a personal profile

Administrator-facing features:

- Add, edit, and safely delete rooms (with a warning before shrinking a room's capacity below what upcoming events need)
- Add, edit, and delete events, and assign a room to an event
- A live dashboard showing total rooms, rooms occupied right now, upcoming events, today's events, total registrations, and capacity warnings
- View and manage all bookings, including cancellation
- View all student registrations, filterable by event or status
- Record and update occupancy after an event, with an automatically calculated occupancy percentage and status badge

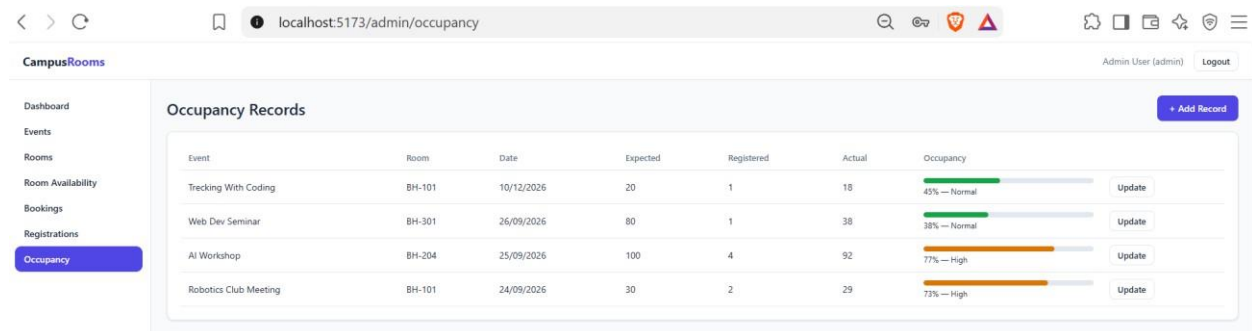
7.2 How Each Feature Works

Room availability checking: given a date, a start time, an end time, and an optional minimum capacity, the system checks every room's status (excluding Maintenance/Inactive rooms), its capacity, and whether any existing room_schedules entry overlaps the requested window, returning two lists — rooms that are available, and rooms that are not, each with a plain-English reason such as “Already booked for ‘AI Workshop’ from 10:00 AM to 12:00 PM.”



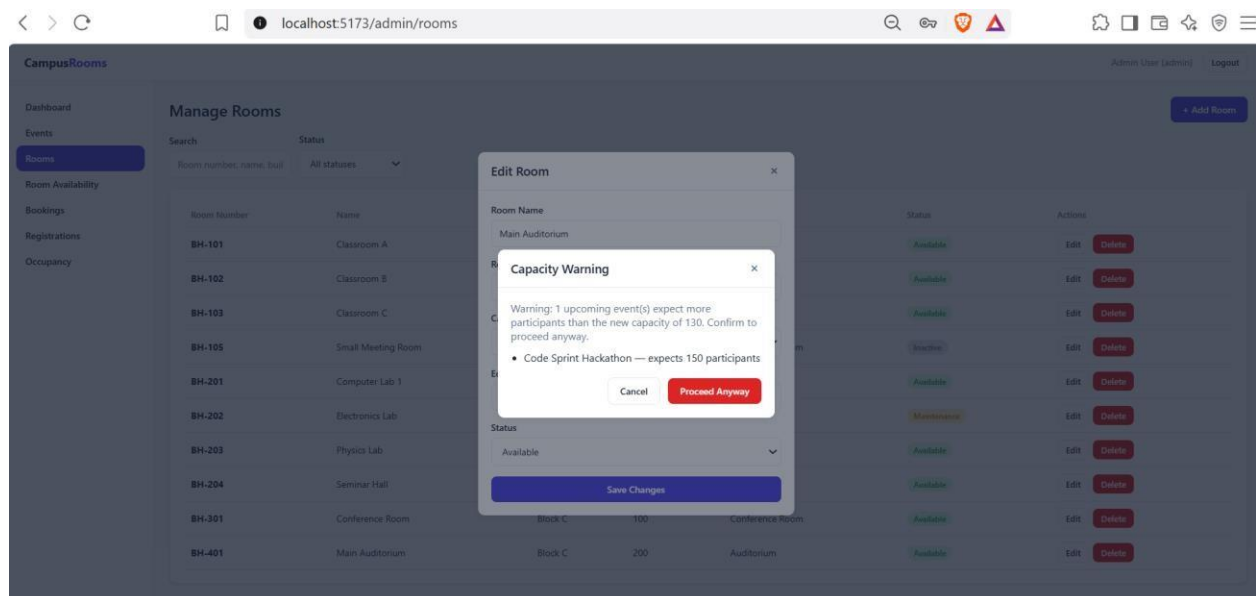
Occupancy percentage calculation: once an admin records the actual headcount for an event, the system computes $(\text{actual_occupancy} \div \text{room_capacity}) \times 100$ and buckets the result into one of

four labels: below 60% is “Normal,” 60–89% is “High,” 90–99% is “Near Capacity,” and 100% or above is “Over Capacity” — shown to the admin as a colored progress bar.



Event	Room	Date	Expected	Registered	Actual	Occupancy	
Trecking With Coding	BH-101	10/12/2026	20	1	18	45% — Normal	Update
Web Dev Seminar	BH-301	26/09/2026	80	1	38	38% — Normal	Update
AI Workshop	BH-204	25/09/2026	100	4	92	77% — High	Update
Robotics Club Meeting	BH-101	24/09/2026	30	2	29	73% — High	Update

Capacity-reduction warning: if an admin tries to reduce a room's capacity below what an upcoming event assigned to that room expects, the system does not silently allow it — it responds with the specific list of affected events and asks the admin to explicitly confirm before the change is applied.



Warning: 1 upcoming event(s) expect more participants than the new capacity of 130. Confirm to proceed anyway.

- Code Sprint Hackathon — expects 150 participants

[Cancel](#) [Proceed Anyway](#)

CHAPTER 8: TESTING

8.1 Postman API Testing

A complete Postman collection was built covering all 22 REST endpoints, organized into six folders (Auth, Rooms, Events, Registrations, Bookings, Occupancy) plus a Health check. The Login requests carry a small test script that automatically saves the returned JWT into a collection variable, and requests such as Create Event, Assign Room, Register, and Create Occupancy Record are chained together using those variables, so a full workflow can be exercised without manually copying any ID or token between requests.

8.2 Integration Testing

Beyond Postman, the system was tested end-to-end against a real, running MySQL database at every stage of development — each backend module was verified with the actual database before the next module was built on top of it, rather than being tested in isolation with mocked data. As a final integration check, the entire database was dropped and rebuilt from `schema.sql` and `seed.sql` from zero, and the full checklist below was re-run against that fresh install to confirm the system works correctly for a first-time setup, not just in an already-running development environment.

8.3 Tools Used for Testing

- Postman — manual and scripted API testing
- curl (command line) — scripted regression testing across every phase of development
- MySQL command-line client — direct verification of table contents and constraint behavior
- Manual browser testing — verifying the React frontend's behavior and appearance

8.4 Test Cases and Results

A representative sample of the test cases executed, drawn from the project's actual test log, is summarized below.

#	Test Case	Expected Result	Actual Result
1	Login with wrong password	401, login rejected	Pass
2	Student token calls an admin-only route	403 Forbidden	Pass
3	Register for the same event twice	Second attempt returns 409 Conflict	Pass
4	Assign a Maintenance-status room to an event	409, room status rejected	Pass
5	Assign a room smaller than expected participants	409, capacity message with exact numbers	Pass
6	Assign a room already booked for an overlapping time	409, conflict message naming the clashing event	Pass
7	Request a room for a slot starting exactly when another ends	No conflict (back-to-back allowed)	Pass

#	Test Case	Expected Result	Actual Result
8	Create a room with negative capacity	400, rejected before reaching the database	Pass
9	Create a room with a duplicate room number + building	409, friendly duplicate message (no raw SQL error)	Pass
10	Reduce a room's capacity below an upcoming event's needs	409 warning listing the affected event(s)	Pass
11	Delete a room with existing booking history	Room marked Inactive instead of deleted	Pass
12	Delete an event with registrations and a room assignment	Cascades cleanly; zero orphaned rows afterward	Pass
13	Create an occupancy record with a negative headcount	400, rejected	Pass
14	Occupancy percentage at 45% / 95% / 108%	Labeled Normal / Near Capacity / Over Capacity	Pass

One test also uncovered a real defect rather than confirming correct behavior: pre-filling the Edit Event form's date/time fields initially used the API's raw UTC-encoded timestamp instead of converting it to the browser's local time, which would have shown an incorrect time (for example, 4:30 AM instead of 10:00 AM) on any machine not running in the UTC timezone. This is discussed further in Chapter 10.

CHAPTER 9: DEPLOYMENT

9.1 Steps to Deploy

The application is deployed locally for development and demonstration as follows:

- Load the schema and seed data: `mysql -u root -p < database/schema.sql`, then ... `< database/seed.sql`
- Start the backend: `cd server, npm install, npm run dev` (listens on port 5000)
- Start the frontend: `cd client, npm install, npm run dev` (listens on port 5173)
- Open `http://localhost:5173` in a browser and log in with a seeded account

9.2 Environment Configuration

All server configuration is kept in a single `.env` file (never committed to version control), covering the database connection (host, port, user, password, database name), the JWT signing secret and token expiry, the port the server listens on, and the frontend's URL for CORS. A `.env.example` template with placeholder values is committed instead, so the required variables are documented without exposing any real credentials.

9.3 Hosting of Frontend and Backend

For this submission, both the frontend and backend run as local Node.js processes and the database runs as a local MySQL instance, which is appropriate for development and in-class demonstration. Vite's development proxy forwards frontend API calls to the backend, avoiding any cross-origin configuration during development. A production deployment was not required for this course submission; Chapter 11 outlines how the same codebase could be adapted to a hosted environment.

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- Ambiguity between ROOM_SCHEDULES and BOOKINGS: the original data model included two tables that could both plausibly track “room + time,” which risked the two falling out of sync if both were updated independently.
- A timezone-conversion bug: the Edit Event form's date/time fields were pre-filled by slicing digits directly out of the API's UTC-encoded timestamp string, rather than converting it to the browser's local time first — correct only by coincidence when the server happens to run in the UTC timezone.
- A MySQL startup failure (“unable to setup unix socket lock file”) encountered mid-development, caused by a stale mysqld process still holding the socket file after an earlier restart.

10.2 Solutions Applied

For the ROOM_SCHEDULES/BOOKINGS ambiguity, a clear rule was adopted and enforced consistently in code: ROOM_SCHEDULES is the single source every conflict check reads from, and BOOKINGS is written at the same time, inside the same database transaction, purely as an audit trail — so the two can never disagree.

For the timezone bug, a small shared utility (dateUtils.js) was written that converts an ISO timestamp to a form field's expected value using JavaScript's local-time Date getters (getHours, getDate, and so on) rather than UTC getters or raw string slicing. The fix was verified by deliberately simulating a non-UTC server timezone (Asia/Kolkata) and confirming the displayed time matched what was actually stored.

For the MySQL startup failure, the stale process was terminated, the leftover socket and PID files were removed, and the service was restarted cleanly — now documented as a known troubleshooting step in the project's README.

10.3 Current Limitations or Known Bugs

- No email verification or password-reset flow currently exists for user accounts.
- Admin accounts can only be created by inserting a row directly into the database — by design, there is no public sign-up path to an admin role.
- Actual occupancy must be entered manually by an admin after an event; there is no automatic check-in mechanism (e.g. QR-code scanning).
- The system has been tested on a single local deployment and has not yet been deployed to a public cloud environment.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

- Email-based password reset and account verification.
- A safe, audited admin UI for promoting an existing user to an admin role, rather than requiring direct database access.
- A calendar view of room schedules, so an admin can see a room's bookings visually across a week or month.
- Email or SMS notifications for registration confirmation and event reminders.
- QR-code based check-in at events, so actual occupancy is captured automatically instead of typed in by hand.

11.2 Possible Integrations or Optimizations

- Integrating a calendar library (such as FullCalendar) on the frontend for the room-schedule view above.
- Caching frequently-requested room-availability queries to reduce repeated database load during peak registration periods.
- Containerizing the application with Docker for a consistent, reproducible deployment environment.
- Adding an automated test suite (e.g. Jest and Supertest) to complement the manual Postman and curl-based testing already performed, so regressions are caught automatically in future changes.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

This project set out to replace informal, error-prone room coordination on a college campus with a single, database-driven web application. Over the course of development, a normalized seven-table MySQL schema, a 22-endpoint Express REST API, and a full React frontend for both students and administrators were designed, built, and tested incrementally, with each layer verified against a real, running database before the next was added.

12.2 What Was Achieved

The finished system enforces role-based authentication; prevents a room from ever being double-booked through one consistently-applied overlap rule; enforces registration rules (no duplicates, capacity-aware limits); gives administrators a live dashboard and full management of rooms, events, bookings, registrations, and occupancy; and was verified with a genuine, documented testing process — including a full fresh-database rebuild and a systematic pass through a final quality checklist — rather than being assumed correct.

12.3 Skills Learned

Working on this project involved practical experience in relational database design and normalization; writing and enforcing constraints at the database layer rather than trusting application code alone; designing a REST API with consistent response conventions and meaningful HTTP status codes; managing multi-table database transactions to keep related data consistent; building a React frontend with Context-based state management; and, importantly, systematic testing — including deliberately trying to break the system's own rules to confirm they hold, which is how the timezone bug described in Chapter 10 was found before it reached a real user.

CHAPTER 13: REFERENCES

- React — Official Documentation, <https://react.dev>
- Vite — Official Documentation, <https://vitejs.dev>
- Node.js — Official Documentation, <https://nodejs.org/en/docs>
- Express.js — Official Documentation, <https://expressjs.com>
- MySQL 8.0 Reference Manual, <https://dev.mysql.com/doc/refman/8.0/en/>
- mysql2 (Node.js MySQL driver) — npm documentation, <https://www.npmjs.com/package/mysql2>
- jsonwebtoken — npm documentation, <https://www.npmjs.com/package/jsonwebtoken>
- bcrypt — npm documentation, <https://www.npmjs.com/package/bcrypt>
- Postman Learning Center, <https://learning.postman.com>
- MDN Web Docs — JavaScript and Web APIs reference, <https://developer.mozilla.org>

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippet

The core room-conflict overlap query, used consistently across the system:

```
// The single overlap rule used everywhere a room's schedule is checked
// (room availability search, event room-assignment, booking creation):
SELECT rs.room_id, rs.start_time, rs.end_time, e.event_name
FROM room_schedules rs
JOIN events e ON rs.event_id = e.event_id
WHERE rs.status = 'Scheduled'
      AND rs.start_time < ?    -- requested end time
      AND rs.end_time   > ?;  -- requested start time
```

Appendix B: Installation and Setup Instructions

- `mysql -u root -p < database/schema.sql`
- `mysql -u root -p < database/seed.sql`
- `cd server && npm install && npm run dev` (backend, port 5000)
- `cd client && npm install && npm run dev` (frontend, port 5173)
- Open `http://localhost:5173` in a browser

Appendix C: Default Login Credentials

All seeded accounts use the password “password123”.

Role	Email
Admin	admin@college.edu
Student	priyanka.k@college.edu
Student	geya.reddy@college.edu
Student (others)	rahul.verma@college.edu, ananya.rao@college.edu, kiran.kumar@college.edu, sneha.patil@college.edu

Appendix D: API Reference Summary

22 endpoints across six resources; full request/response examples are in the accompanying Postman collection.

Resource	Endpoints
Auth	POST /register, POST /login, GET /me
Rooms	GET /, GET /:id, GET /available, POST /, PUT /:id, DELETE /:id
Events	GET /, GET /:id, POST /, PUT /:id, DELETE /:id, PUT /:id/assign-room
Registrations	POST /, GET /my, GET /event/:eventId, GET /
Bookings	GET /, POST /, PUT /:id, DELETE /:id
Occupancy	GET /, POST /, PUT /:id

Appendix E: Screenshots

The following pages are left for the student to complete with actual screenshots taken from the running application before submission. Suggested screenshots, in order:

1. Login Page

localhost:5173/login

Log in

Welcome back to the Event & Room Occupancy Management System.

Email

Password

[Log In](#)

Don't have an account? [Register here](#)

2. Student Dashboard

localhost:5173/student

CampusRooms Kiran Kumar (student) Logout

Dashboard Events My Registrations Room Availability Profile

Welcome, Kiran Kumar

3 Upcoming Events

3 My Active Registrations

Upcoming Events

[Browse all](#)

Faculty Planning Meet
Sep 29, 04:30 PM - Room not assigned

Chess Championship
Oct 2, 03:30 PM - BH-102

Trecking With Coding
Dec 10, 03:30 PM - BH-101

My Registrations

[View all](#)

Trecking With Coding [Registered](#)

Code Sprint Hackathon [Registered](#)

Robotics Club Meeting [Attended](#)

3. Events Page (student's view)

localhost:5173/student/events

Dashboard Events My Registrations Room Availability Profile

Events

Search Type All types

Robotics Club Meeting
[Club Event](#)

Sep 24, 2026
09:30 PM - 11:00 PM

Expected: 30
Room: BH-101 (capacity 30)

[View Details](#) [Registered](#)

AI Workshop
[Workshop](#)

Sep 25, 2026
03:30 PM - 05:30 PM

Expected: 100
Room: BH-204 (capacity 120)

[View Details](#) [Event ended](#)

Web Dev Seminar
[Seminar](#)

Sep 26, 2026
07:30 PM - 09:30 PM

Expected: 80
Room: BH-301 (capacity 100)

[View Details](#) [Event ended](#)

Code Sprint Hackathon
[Hackathon](#)

Sep 27, 2026
02:30 PM - 02:30 PM

Expected: 150
Room: BH-401 (capacity 200)

[View Details](#) [Registered](#)

Faculty Planning Meet
[Meeting](#)

Sep 29, 2026
04:30 PM - 05:30 PM

Expected: 15
Room: Not assigned

[View Details](#) [Register](#)

Chess Championship
[Competition](#)

Oct 02, 2026
03:30 PM - 10:30 PM

Expected: 40
Room: BH-102 (capacity 60)

[View Details](#) [Register](#)

Trecking With Coding
[Competition](#)

Dec 10, 2026
03:30 PM - 04:30 PM

Expected: 20
Room: BH-101 (capacity 30)

[View Details](#) [Registered](#)

4. Room Availability Page

localhost:5173/student/rooms/availability

CampusRooms

Dashboard Events My Registrations Room Availability Profile

Room Availability

Select a date and time range to see which rooms are free and which are already booked.

Date: 26-09-2026 Start time: 10:00 End time: 12:00 Required capacity (optional): Any

Check Availability

✓ Available (8)

- BH-101 — Classroom A**
Block B · Capacity 30 · Classroom · Whiteboard, Projector
- BH-102 — Classroom B**
Block B · Capacity 60 · Classroom · Whiteboard, Projector, AC
- BH-103 — Classroom C**
Block B · Capacity 60 · Classroom · Whiteboard, Projector
- BH-201 — Computer Lab 1**
Block B · Capacity 60 · Lab · Computers, Projector, AC
- BH-203 — Physics Lab**

✗ Unavailable (2)

- BH-105 — Small Meeting Room**
Room is currently marked as Inactive.
- BH-202 — Electronics Lab**
Room is currently marked as Maintenance.

5. Admin Dashboard

localhost:5173/admin

CampusRooms

Admin User (admin) Logout

Admin Dashboard

10 Total Rooms

7 Available Right Now

1 Occupied Right Now

3 Upcoming Events

1 Today's Events

13 Total Registrations

Upcoming Events

Event	Start	Room
Faculty Planning Meet	Sep 29, 04:30 PM	Not assigned
Chess Championship	Oct 2, 03:30 PM	BH-102
Tracking With Coding	Dec 10, 03:30 PM	BH-101

[View all events](#)

Capacity Warnings

No capacity problems detected. The system also prevents new scheduling conflicts automatically at the point a room is assigned, so none can exist here.

6. Manage Rooms page, including the Add/Edit Room modal

localhost:5173/admin/rooms

CampusRooms

Admin User (admin) Logout

Manage Rooms

Search: Room number, name, building Status: All statuses

Room Number	Name	Building	Capacity	Type	Status	Actions
BH-101	Classroom A	Block B	30	Classroom	Available	Edit Delete
BH-102	Classroom B	Block B	60	Classroom	Available	Edit Delete
BH-103	Classroom C	Block B	60	Classroom	Available	Edit Delete
BH-105	Small Meeting Room	Block B	20	Conference Room	Inactive	Edit Delete
BH-201	Computer Lab 1	Block B	60	Lab	Available	Edit Delete
BH-202	Electronics Lab	Block B	40	Lab	Maintenance	Edit Delete
BH-203	Physics Lab	Block B	40	Lab	Available	Edit Delete
BH-204	Seminar Hall	Block B	120	Seminar Hall	Available	Edit Delete
BH-301	Conference Room	Block C	100	Conference Room	Available	Edit Delete
BH-401	Main Auditorium	Block C	200	Auditorium	Available	Edit Delete

[Add Room](#)

7. Manage Events page, including the Assign Room modal

The screenshot shows the 'Manage Events' page in the CampusRooms application. The page has a sidebar with navigation links: Dashboard, Events (selected), Rooms, Room Availability, Bookings, Registrations, and Occupancy. The main content area is titled 'Manage Events' and includes a search bar and a table of events. The table has columns for Event, Type, Start, Expected, Room, and Actions. The events listed are:

Event	Type	Start	Expected	Room	Actions
Robotics Club Meeting Robotics Club	Club Event	Sep 24, 09:30 PM	30	BH-101	Edit Assign Room Delete
AI Workshop CS Department	Workshop	Sep 25, 03:30 PM	100	BH-204	Edit Assign Room Delete
Web Dev Seminar IT Department	Seminar	Sep 26, 07:30 PM	80	BH-301	Edit Assign Room Delete
Code Sprint Hackathon Coding Club	Hackathon	Sep 27, 02:30 PM	150	BH-401	Edit Assign Room Delete
Faculty Planning Meet Administration	Meeting	Sep 28, 04:30 PM	15	Not assigned	Edit Assign Room Delete
Chess Championship Sports Committee	Competition	Oct 2, 03:30 PM	40	BH-102	Edit Assign Room Delete
Tracking With Coding QA	Competition	Dec 10, 03:30 PM	20	BH-101	Edit Assign Room Delete

8. Manage Occupancy page, showing the colored percentage bar

The screenshot shows the 'Occupancy Records' page in the CampusRooms application. The page has a sidebar with navigation links: Dashboard, Events, Rooms, Room Availability, Bookings, Registrations, and Occupancy (selected). The main content area is titled 'Occupancy Records' and includes a table of occupancy records. The table has columns for Event, Room, Date, Expected, Registered, Actual, Occupancy, and Actions. The records listed are:

Event	Room	Date	Expected	Registered	Actual	Occupancy	Actions
Tracking With Coding	BH-101	10/12/2026	20	1	18	60% — High	Update
Web Dev Seminar	BH-301	26/09/2026	80	1	38	38% — Normal	Update
AI Workshop	BH-204	25/09/2026	100	4	92	77% — High	Update
Robotics Club Meeting	BH-101	04/09/2026	30	2	29	97% — Near Capacity	Update